1 Introduction

Collaborative filtering is commonly used for recommender systems. Provided with a dataset of historical user preferences, this report explores our implementation of collaborative filtering through `spark`'s API. Specifically, alternating least squares(ALS) method is used to create our model.

1.1 Collaborative Filtering

A recommender system refers to a model that is able to predict the future preference of a user. Collaborative filtering (CF) is a method to construct a recommender system. The main idea of CF is that similar users will share similar interests across users.

Given a matrix $R \in R^{m \times n}$ of ratings given by m users for n items, we can factorize matrix R into two components $U \in R^{m \times k}$ and $V \in R^{k \times n}$.

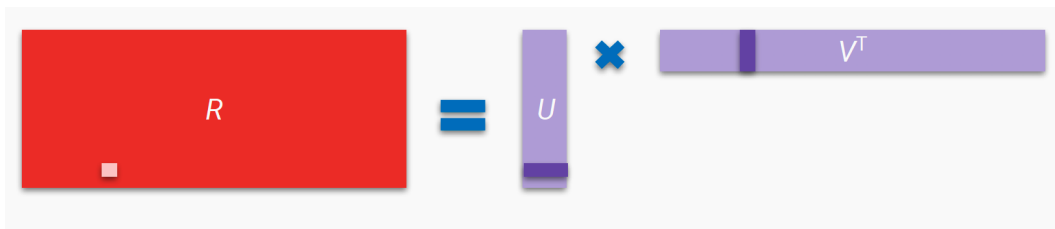

Figure 1: Matrix R factorization

To predict R_{ij} (user i ratings on item j), it is the dot product of $< U_i, V_j >$. Thus the goal is to optimize for the factored matrices U and V.

Ideally, we want to minimize

$$\min_{U,P} \sum_{i,j} (R_{i,j} - < U_i, V_j >)^2$$

1.1.1 Alternating Least Squares

With two parameters to optimize, `spark`'s implementation of CF uses alternating least squares (ALS). We optimize U by fixing P constant and then optimize P by fixing U. We repeat the process until convergence.

1.1.2 Evaluation

The metric we used to evaluate our ranking system is precision: precision at k and Mean Average Precision (MAP).

Precision is the fraction of the predicted items that are in the set of validation items. Precision at k measures the precision of the first k items whereas MAP measures the precision of all the items.

1.2 Data Preview

The dataset we will be working contains the implicit feedback provided from each user. We are provided with training, validation and test dataset where each contains 1 million, 10k, and 100k users histories respectively.The links can be found through `hdfs:/user/bm106/pub/project`

The following is a sample of the raw input data.

```
+--------------------+-----+------------------+-----------------+
|             user_id|count|          track_id|__index_level_0__|
+--------------------+-----+------------------+-----------------+
|b80344d063b5ccb32...|    1|TRIQAUQ128F42435AD|                0|
|b80344d063b5ccb32...|    1|TRIRLYL128F42539D1|                1|
|b80344d063b5ccb32...|    2|TRMHBXZ128F4238406|                2|
+--------------------+-----+------------------+-----------------+
```

Figure 2: Input data sample

We are also provided with four additional dataset that could potentially improve our model.

2 Baseline Model

This section will go through our implementation of `spark`'s ALS method to build our baseline model. The code for this baseline model can be found here.

2.1 Hyper Parameters

The hyper-parameters we tune in `spark`'s ALS model to optimize the performance on the validation set are:

1. `rank` - the dimension of our latent factor
   (a) We tuned this parameter within the [2,5,8,10] range
2. `regParam` - regularization parameter in ALS
   (a) We tuned this parameter within the [0.001,0.01,0.1] range
3. `alpha` - scaling parameter to the implicit feedback data.
   (a) We tuned this parameter within [0.1,0.3,0.7,1] range.

The other parameters that we used to train our ALS model are: `numBlocks`: 10 (default), `maxIter`: 10 (default), `implicitPrefs`: True (specify to use implicit feedback modeling).

2.2 Results

For each Cartesian product of the above hyper-parameters range we picked, we will evaluate it using both precision at k and MAP.

The following table summarizes the score for the subset of the hyper-parameters we tuned.

regParam, rank, alpha	Precision @ 500	MAP
0.001, 5, 0.3	0.00417	0.00265
0.001, 5, 0.7	0.00376	0.01136
0.001, 5, 1.0	0.00367	0.01176
0.001, 8, 0.3	0.00411	0.0125
0.001, 8, 0.7	0.00387	0.01299
0.001, 8, 1.0	0.00395	0.01351
0.001, 10, 0.3	0.00430	0.01563
0.001, 10, 0.7	0.00396	0.01587
0.001, 10, 1.0	0.00398	0.01639
0.01, 2, 0.1 (*)	0.01042	0.03357
0.01, 5, 0.3	0.00409	0.00266
0.01, 5, 0.7	0.00391	0.01299
0.01, 5, 1.0	0.00383	0.01190
0.01, 8, 0.3	0.00451	0.0125
0.01, 8 , 0.7	0.00418	0.01298
0.01, 8, 1.0	0.00405	0.01351
0.01, 10, 0.3	0.00435	0.01563
0.01, 10, 0.7	0.00390	0.01587
0.01, 10, 1.0	0.00390	0.01639
0.1, 5, 0.3	0.00427	0.00265
0.1, 5, 0.7	0.00386	0.01136
0.1, 5, 1.0	0.00381	0.01176
0.1, 8, 0.3	0.00452	0.0125
0.1, 8, 0.7	0.00418	0.01282
0.1, 8, 1.0	0.00414	0.01351
0.1, 10, 0.3	0.00431	0.01538
0.1, 10, 0.7	0.00396	0.01563
0.1, 10, 1.0	0.00388	0.01613

After optimizing our hyper-parameters, we perform our model on the test set. The precision at 500 for the test set is 0.0104717. The MAP for the test set is 0.03477.

2.3 Discussion

For each combination of the hyper-parameters, the score for the precision for the first 500 recommended items fluctuates around 0.003 to 0.004 and the MAP metric is around 0.002 to 0.01. The best combination of the hyper-parameters are: `regParam`: 0.01, `rank`: 2, `alpha`: 0.1, where we achieved the MAP at 0.03357 on the validation set. The performance of our model doesn't perform very well, the reason may be the lack of the useful information in the training samples. Or there is too much noise in the data.

2.4 Precision at 20

Instead of measuring the precision for the first 500 items, a more realistic music recommending system would only recommend roughly 20 songs to a user. With that, we ran our model with

the best parameter and evaluate with precision at 20. Precision at 20 is 0.01124 while MAP is 0.00718. The results improved.

3 Extension

The above `spark`'s ALS model used a particular form of implicit-feedback modeling[1]. Implicit Feedback are feedback we observe from users, in our case, the count column in our raw data input. These signals are usually extremely weak. Suppose a user listened to a specific track with count of 1. That particular user may not specifically liked the song, compared to a user with count 30 to a track. In this extension, we assume the small counts (counts less than x) are weak signals and we consider ignoring them in our training procedure. Other than that, we also tried to take the log compression on the counts, and predict the percentage change on the counts instead. We found that the results got slightly worse if drop the low counts, and it gets better if we take log of the counts. The code for this extension can be found here.

3.1 Results

We found that after we dropped the counts that are smaller than 1, the MAP on the test set becomes 0.0259 and precision is 0.0029. The reason that it gets worse may be by dropping those observations, we get less training samples.

If we take the log compression on the output, the test MAP on the test set becomes 0.036 and precision becomes 0.0023, which performs better than baseline model.

3.2 Discussion

A major compromise we made in order achieve the above result is through down-sampling. We achieved the above results by training on only on 5% of the dataset. The compromise must be made because the runningtime of our algorithm on the full dataset would take too long for dumbo cluster to process.

4 Member Distribution

Nopphrakorn and Xiao worked together on the coding development and producing the final report.

[1] More information about how `spark`'s ALS uses this implicit-feedback modeling here

References

[1] *Apache Spark (2.4.3)* Retrieved from https://spark.apache.org/docs/latest/index.html

[2] *Big Data Week 13 - Recommender System Slides*